

# spinalSynth MIDI PICO2

---

A standalone, real-time hardware USB MIDI-to-UART bridge implemented on the **Raspberry Pi Pico 2 (RP2350)** using the official Pico SDK and TinyUSB.

The Pico 2 acts as a **USB MIDI Host** to interface with multiple physical MIDI devices (via a USB Hub) and stream UART register update frames directly to the spinalSynth hardware.

---

## Hardware Configuration & Pinout

### 1. Connecting MIDI Controllers

- Because the Pico 2 has a single USB port, you must connect a standard **USB Hub** to the Pico 2's USB-C port using an OTG adapter cable.
- Plug your MIDI controllers (e.g. keyboard, CC controller) directly into the USB Hub. The Pico 2 will supply host power and enumerate all controllers concurrently.

### 2. Wiring to spinalSynth (UART) & Debug Console

Connect the Pico 2 UART TX pins directly to the spinalSynth RX pins (3.3V logic levels). A secondary UART port is used for debugging console logs to a PC:

| Pico 2 Pin            | Function        | Target / Connection                                                         |
|-----------------------|-----------------|-----------------------------------------------------------------------------|
| <b>GPIO 0 (Pin 1)</b> | UART0 TX        | spinalSynth RX (Command Input)                                              |
| <b>GPIO 1 (Pin 2)</b> | UART0 RX        | spinalSynth TX (Optional Command Output)                                    |
| <b>GPIO 2 (Pin 4)</b> | Input (Pull-up) | Mode Selector Jumper (Open/default = Legato/Stacked, GND/Pin 3 = Retrigger) |
| <b>GPIO 4 (Pin 6)</b> | UART1 TX        | Debug Console RX (115200 Baud serial monitor)                               |
| <b>GPIO 5 (Pin 7)</b> | UART1 RX        | Debug Console TX (Optional serial command input)                            |
| <b>GND</b>            | Ground          | Common Ground                                                               |

*Note: Since the onboard USB-C port is configured in USB Host mode to talk to the MIDI Hub/devices, debugging stdout cannot run over USB CDC. It is redirected to UART1 (Pins GP4/GP5) at 115200 baud.*

---

## MIDI to Register Mapping

All parameters are mapped sequentially across CC 1 to 12:

| MIDI Event | Target Register      | Address          | Description                                             |
|------------|----------------------|------------------|---------------------------------------------------------|
| Note ON    | DDS Frequency / Gate | 0x00-0x02 / 0x45 | Set 24-bit DDS tuning word; writes 0x01 to ENV_GATE     |
| Note OFF   | Gate                 | 0x45             | Writes 0x00 to ENV_GATE                                 |
| CC 1       | PWM_WIDTH            | 0x04             | Duty cycle for PWM (scaled 0-127 $\rightarrow$ 0-255)   |
| CC 2       | WAVE_SEL             | 0x03             | Waveform selection (maps onto states 0 to 5)            |
| CC 3       | ENV_ATTACK           | 0x41             | Attack time duration                                    |
| CC 4       | ENV_DECAY            | 0x42             | Decay time duration                                     |
| CC 5       | ENV_SUSTAIN          | 0x43             | Sustain Level                                           |
| CC 6       | ENV_RELEASE          | 0x44             | Release time duration                                   |
| CC 7       | ENV_CTRL [2]         | 0x40             | Loop Mode (ON if $\geq$ 64, OFF if < 64)                |
| CC 8       | ENV_CTRL [5:4]       | 0x40             | Curve Select (0=Lin, 1=Exp, 2=Log, 3=S-Curve)           |
| CC 9       | FILTER_ENABLE        | 0x50             | Filter Enable (ON if $\geq$ 64, OFF if < 64)            |
| CC 10      | FILTER_MODE          | 0x51             | Response Mode (LP if 0-42, BP if 43-85, HP if 86-127)   |
| CC 11      | FILTER_CUTOFF        | 0x52             | Cutoff Frequency (scaled 0-127 $\rightarrow$ 0-255)     |
| CC 12      | FILTER_RESONANCE     | 0x53             | Resonance / Feedback (scaled 0-127 $\rightarrow$ 0-255) |
| CC 64      | VOLUME               | 0x05             | Output volume (scaled 0-127 $\rightarrow$ 0-255)        |

## Monophonic Playback Modes

The bridge supports two monophonic note priority and envelope gate behaviors, selected via a hardware jumper on **GPIO 2 (Pin 4)**:

### 1. Legato Mode (Stacked) — *Default Behavior (Jumper Open)*

- **Selection:** Leave the selector pin **open** (GP2 is pulled High internally).
- **MS-20 Style Behavior:**
  - Pressing a note sets the pitch and sends **GATE ON** (0x01 to REG\_ENV\_GATE).
  - While holding a note, pressing additional notes updates the pitch to the newest note, but the envelope gate remains high (legato play).
  - Releasing a note while other keys are still held down reverts the pitch back to the most recently pressed remaining note, without interrupting the gate.
  - **GATE OFF** (0x00) is only sent when all keys are released (the note stack is empty).

### 2. Retrigger Mode — *Jumper Installed (GP2 connected to GND)*

- **Selection:** Install a jumper bridging **GP2 (Pin 4)** and **GND (Pin 3)**.
- **Standard Monophonic Behavior:**
  - Every new Note On event immediately updates the pitch and sends **GATE ON**.
  - Every Note Off event immediately sends **GATE OFF**, stopping the sound even if other keys are still held.

## Technical Details & USB Host Stack

- **Native USB Controller (Port 0):** This project runs on the native hardware USB controller of the RP2350, utilizing standard differential signals. This keeps the implementation clean and avoids the complexity, core dedication, clock-

scaling, and resource consumption of bit-banged PIO solutions.

- **TinyUSB Class Driver Callback:** In Pico SDK 2.x, the MIDI Host class driver is not registered automatically. To bind and mount the USB MIDI device interface when plugged in, the application implements the weak callback `usbh_app_driver_get_cb()` to return the MIDI class driver:

```
usbh_class_driver_t const* usbh_app_driver_get_cb(uint8_t* driver_count) {
    static usbh_class_driver_t const midi_driver = {
        .name = "MIDI",
        .init = midih_init,
        .deinit = midih_deinit,
        .open = midih_open,
        .set_config = midih_set_config,
        .xfer_cb = midih_xfer_cb,
        .close = midih_close
    };
    *driver_count = 1;
    return &midi_driver;
}
```

Implementing this callback registers the driver and resolves device-recognition/mounting issues.

---

## Build Instructions

Using the official Raspberry Pi Pico VS Code Extension:

1. Open the `/spinalSynth_MIDI_PIC02` project folder in VS Code.
2. The extension will automatically configure the CMake environment for the **RP2350** processor.
3. Click the **Compile** button in the VS Code status bar (or run **CMake: Build** from the command palette).
4. Put your Pico 2 in bootSEL mode, and copy the compiled `.uf2` file to the drive (or click **Flash** with a debugger connected).